

Lauffähiger Code

<https://github.com/yaniktf/iv-repo>, Ordner 11/.

Ausführung

Anders als in den vorigen Übungen reicht `elm reactor` hier nicht: `elm reactor` kompiliert nur die `.elm`-Datei und liefert dafür eine eigene, generierte HTML-Seite aus – unser `index.html` mit dem Dagre-Script-Tag und dem Ports-Glue-Code wird dabei komplett ignoriert. Ohne diesen JS-Code läuft `dagre.layout` nie, und es erscheinen nur die Überschriften, aber keine Graphen.

Stattdessen `Main.elm` bauen und den Ordner mit einem einfachen HTTP-Server ausliefern:

```
cd 11
elm make Main.elm --output=main.js
python3 -m http.server 8731
```

Anschließend im Browser `http://localhost:8731/index.html` öffnen.

Zusammenspiel Elm / JavaScript (Ports)

Elm kennt Dagre nicht direkt; die Layout-Berechnung läuft über zwei Ports, wie im Elm-Guide-Kapitel zu Ports beschrieben und in `madsflensted/elm-dagre-example` vorgeführt:

```
port graphs : (Value -> msg) -> Sub msg
port layout : String -> Cmd msg
```

Elm schickt über `layout` eine JSON-Beschreibung des Graphen (Knoten/Kanten inkl. geschätzter Breite/Höhe) an JavaScript. In `index.html` liest das Dagre selbst ein, berechnet das Layout und schickt das Ergebnis (jetzt mit `x/y`-Positionen und Kanten-Punkten) über `graphs` zurück an Elm:

```
app.ports.layout.subscribe(function (data) {
  var parsed = JSON.parse(data);
  var g = dagre.graphlib.json.read(parsed);
  dagre.layout(g);
  var graph = dagre.graphlib.json.write(g);
  graph.key = parsed.key;
  app.ports.graphs.send(graph);
});
```

Da mehrere Layout-Anfragen gleichzeitig unterwegs sein können (Satz-Graph plus drei Ranker-Varianten für Aufgabe 11.3), bekommt jede Anfrage ein `key`-Feld, das JavaScript unverändert zurückspiegelt. Elm ordnet die Antwort darüber wieder der richtigen Anfrage zu (`sentenceGraph` bzw. `Dict String GraphData` für die Ranker-Varianten).

Ausgangspunkt war das erweiterte Minimal-Beispielprogramm (TypedSvg statt Svg) aus der Vorlesung; die Knotengröße wird nicht mehr fest vorgegeben, sondern vor dem Senden an Dagre aus der Label-Länge geschätzt (`estimateNodeSize`), damit z. B. "sentence" eine breitere Box bekommt als "is".

Aufgabe 11.1: Sentence Tokenization (ohne CSS-Klassen)

Die Kantenrichtung wird über einen SVG-marker sichtbar gemacht: Ein `defs`-Block definiert einen Pfeilkopf, der jeder `polyline` per `markerEnd "url(#arrowhead)"` angehängt wird.

```
TypedSvg.marker
  [ SA.id "arrowhead"
  , SA.markerWidth (Px 8)
  , SA.markerHeight (Px 8)
  , SA.refX "6"
  , SA.refY "3"
  , SA.orient "auto"
  ]
  [ TypedSvg.path [ SA.d "M0,0 L6,3 L0,6 Z", SA.fill (Paint Color.black) ] [] ]
```

CSS-Klassen der Knoten werden hier laut Aufgabenstellung noch nicht berücksichtigt – alle Knoten werden einheitlich (weißer Hintergrund, blauer Rahmen) gezeichnet.

Aufgabe 11.2: Sentence Tokenization (mit CSS-Klassen)

Derselbe Graph wird ein zweites Mal gezeichnet, jetzt mit den Original-CSS-Klassen aus der Demo (`type-TOP`, `type-S`, ..., `type-TK` für die Token-Blätter). Jeder Knoten trägt seine Klasse als `class`-Attribut auf der `g`-Gruppe; ein per `TypedSvg.style` eingebettetes `<style>`-Element im SVG übernimmt die Original-Regel der Demo:

```
g.type-TK > rect { fill: #00ffd0; }
```

Dadurch werden die Token-Blätter ("This", "is", "an", "example", "sentence") türkis eingefärbt, alle anderen Knoten bleiben weiß – exakt wie im Original-Dagre-D3-Demo, nur dass Elm über CSS-Klassen und nicht über eine eigene Fallunterscheidung im Elm-Code färbt.

Aufgabe 11.3: Folie 456 – drei Ranker-Varianten

Für diese Aufgabe wird der Graph von Folie 456 (15 Knoten, 0–14, 26 gerichtete Kanten) dreimal mit unterschiedlicher Dagre-ranker-Konfiguration (`network-simplex`, `tight-tree`, `longest-path`) im `value`-Feld der Layout-Anfrage an Dagre geschickt und untereinander ausgegeben:

```
jsonGraphValues rankdir ranker =
  object
    [ ...
    , ( "ranker", Encode.string ranker )
    ]
```

Der Graph wird dabei unverändert (im Ausgangszustand von Folie 456, vor der händischen Kreisentfernung/Schichtung, die auf der Folie nur zur Illustration gezeigt wird) an Dagre übergeben – Kreisentfernung, Rangzuweisung, Ordnen innerhalb der Ränge und Koordinatenzuweisung übernimmt **dagre.layout** vollständig selbst. Genau das macht den Unterschied zwischen den drei Ranker-Varianten sichtbar: Knoten 12 (eine Senke ohne ausgehende Kanten) landet bei **network-simplex** und **tight-tree** in einem mittleren Rang, bei **longest-path** dagegen ganz unten, weil dieser Ranker jeden Knoten so spät wie möglich einordnet.